KATUP

Kendali Antrian Titik Urai Kemacetan Perkotaan



COURSE LECTURER :

Prof. Dr. Ir. Riri Fitri Sari, M.M., M.Sc.

GROUP 03 :

Muhammad Hashif Jade (2406396786)

Muhamad Rifqi Fadil Itsnain (2406355306)

Zulfahmi Fajri (2406345425)

REKAYASA PERANGKAT LUNAK - 01

COMPUTER ENGINEERING

FACULTY OF ENGINEERING - UNIVERSITAS INDONESIA

2026

Chapter 1. Introduction

1. Introduction and Background

Traffic jams are a common problem in Jakarta. One cause is a road that becomes narrower. This can happen because of road repairs, construction work, accidents, illegal parking, or activities on the road. When there is less space for vehicles, not all vehicles can pass through quickly. As a result, a queue forms before the narrow part of the road.

If the queue keeps growing, it can reach the previous intersection. Vehicles from other directions may then have difficulty crossing that intersection. A traffic jam that starts on one road can spread to nearby roads.

2. Project Definition

To address this problem, we designed KATUP, which stands for Kendali Antrean Titik Urai Kemacetan Perkotaan. KATUP changes traffic control based on road conditions. Its goal is to prevent a queue at a narrow road from reaching the previous intersection. It works in two ways: by changing the traffic light timing at the intersection and by moving some vehicles to another route. We call the intersection before the narrow road the control point. At this intersection, KATUP controls how many vehicles enter the road toward the bottleneck.

KATUP uses three agents that work together:

- Observation Agent: This agent reads traffic data from the simulation. It checks the road between the intersection and the bottleneck. It also checks the vehicles waiting before the traffic light. The data includes the number of vehicles, queue length, and average speed. The agent reports a risk when the queue from the bottleneck reaches 80 percent of the distance to the previous intersection. We use 80 percent as a starting value. We may change it based on the test results.
- Valve Agent: This agent controls the traffic light in two modes. In normal mode, the light uses its usual timing. When the Observation Agent reports that the queue has reached the risk

level, the Valve Agent starts the bottleneck control mode. In this mode, it gradually reduces the green time for vehicles going toward the narrow road. Those vehicles must then wait longer before entering. The green time still has minimum and maximum limits. The Valve Agent also checks the queue before the traffic light. If that queue becomes too long, it will not reduce the green time further. The system returns to normal mode when the queue from the bottleneck stays below 60 percent for three traffic light cycles in a row.

- Route Agent: This agent checks whether another road can take some of the vehicles when a bottleneck occurs. It receives information from the Observation Agent and compares other routes based on traffic levels and estimated travel time. It limits the number of vehicles sent to another route so that the other road does not become too crowded. A digital road sign in the simulation shows the selected route. The program changes the routes of some vehicles that can still take the other road before they pass the intersection.

The three agents work in a repeated cycle. The Observation Agent checks the road, the Valve Agent controls the traffic light, and the Route Agent selects another route. After the Valve Agent and Route Agent take action, the Observation Agent checks the road again. The result is used for the next decision.

We will test KATUP under three conditions:

1. Normal traffic without a bottleneck.
2. Traffic with a bottleneck but without KATUP.
3. Traffic with a bottleneck and with KATUP.

We will record vehicle waiting time, queue length, the number of vehicles that pass the bottleneck, and how often the queue reaches the previous intersection.

3. User Story

- As a driver, I want to see information about another route on a digital road sign when the road ahead becomes narrow. This can help me avoid a long queue.

- As a traffic operator, I want a system that can detect a growing queue and change the traffic light timing at the previous intersection. This can help prevent the queue from blocking the intersection.

4. Software and Hardware Required

We are making KATUP as a simulation. We use laptops to run the program. This prototype does not use real cameras, road sensors, traffic lights, or digital road signs.

We use SUMO to create the roads, intersections, vehicles, traffic lights, bottleneck, and other routes. We use Python to build the three agents. The Python program connects to SUMO through TraCI. This lets the agents read traffic conditions and change traffic lights and vehicle routes while the simulation is running. The information on the digital road sign is also shown in the simulation.

If KATUP is tested on a real road in the future, it will need equipment to read traffic conditions, traffic lights that can be controlled, information signs, and a communication network. Testing on a real road is outside the scope of our current project.

5. Competitor Product

Google Maps and Waze help drivers find routes based on traffic conditions. ATCS and ITCS are used to monitor or control traffic at intersections. These systems are related to the problem we study, but KATUP focuses on the queue caused by one road bottleneck.

In this project, we want to see whether changing traffic light timing and moving some vehicles to another route can stop the queue from reaching the previous intersection. We will test this idea through a simulation.

6. Risk Analysis of System Development

We see several problems that may occur while we build KATUP:

- The queue measurement may be wrong: If the Observation Agent reads the queue length incorrectly, the traffic light may change too early or too late. We will also check the number of vehicles and their average speed as extra data.

- A new queue may form before the traffic light: Reducing the green time may cause vehicles to build up before the intersection. The Valve Agent will check this queue. It will stop reducing the green time if the queue becomes too long. The yellow light and the safe time between directions will stay the same.
- The other route may become crowded: Sending too many vehicles to another road may cause a traffic jam there. The Route Agent will move vehicles in stages and check the road before moving more vehicles.
- Data between agents may be lost: If data is not available, the program will not change the traffic light or vehicle routes based on old data. It will record the error. We will then test how to restart the simulation or reconnect the program.

KATUP will only be tested in a simulation. The results may show how the system works and whether it could be useful. They will not prove that KATUP is ready to be installed on a real road.

Chapter 2. Project Management Plan

1. Project Management Overview

In this chapter, we explain how we plan to build the KATUP prototype during one semester. We describe the work, team roles, resources, schedule, use of Jira, and possible risks. This plan follows the goals in Chapter 1: checking queues caused by a bottleneck, controlling the traffic light, and moving some vehicles to another route.

We divide the work into weekly targets so that we can check our progress step by step. Each week, we choose tasks from Jira, work on them, and review the results together. We can show completed work during our class demonstrations. If a test shows a problem, we will change the system rules or adjust our schedule.

2. Organization and Roles

We are three Computer Engineering students in Group 3. We divide the work as follows:

- **Muhamad Rifqi Fadil Itsnain - Project and Simulation Coordinator.** Rifqi manages the schedule and tasks in Jira. He builds the road model and bottleneck scenario in SUMO. He also sets up the connection between SUMO and Python through TraCI.
- **Muhammad Hashif Jade - Observation and Valve Agent Developer.** Hashif builds the queue reading function, sets the first rules for detecting a risk, and develops the traffic light control function. He also checks whether changing the traffic light creates a long queue before the intersection.
- **Zulfahmi Fajri - Route Agent and Testing Developer.** Fahmi builds the rules for choosing another route and deciding which vehicles will use it. He shows route information on the digital road sign in the simulation. He also prepares the tests and records their results.

To make our responsibilities clear, we show them in the RACI matrix in Table 2.1. R means the person does the work, A means the person is responsible for the result, C means the person gives input, and I means the person is kept informed.

Table 2.1. Team Responsibility Matrix (RACI)

Work	Rifqi	Hashif	Fahmi
Plan the schedule and set up Jira	A / R	C	C
Design the SUMO road network (.net.xml)	A / R	C	I
Create traffic flow and the bottleneck scenario	A / R	I	C
Develop the Observation Agent	C	A / R	I
Develop the Valve Agent	I	A / R	C
Develop the Route Agent	I	C	A / R
Connect Python and SUMO through TraCI	A / R	R	R
Make the digital road sign display	I	I	A / R
Test the scenarios and collect data	R	R	A / R
Write the report and prepare class demonstrations	A / R	R	R

We planned this work based on the two user needs in Chapter 1. Drivers need information about another route when a bottleneck occurs. We address this need through the Route Agent and the digital road sign display. Traffic operators need a system that can check queues and control the traffic light before a queue reaches the intersection. We address this need through the Observation Agent and the Valve Agent.

Before writing the program, we will define the rules for each agent. These rules include how we measure a queue, when the traffic light changes, the limits on traffic light timing, which vehicles can take another route, and when route changes should stop. We will also decide what data we need to record to check whether the two user needs are met.

3. Work Breakdown Structure (WBS)

We divide the KATUP project into five parts:

1. Planning and requirements: Finish Chapters 1 and 2, set up Jira and a GitHub repository, write the user needs, and define the first rules and the flow of information between the three agents.
2. Simulation: Build the roads, intersection, traffic lights, bottleneck, other routes, and vehicles in SUMO. We will prepare both normal road conditions and conditions with a bottleneck.
3. Agents: Build the Observation Agent to read traffic conditions, the Valve Agent to control the traffic light, and the Route Agent to move some vehicles to another route. We will connect the agents to SUMO using Python and TraCI.
4. Testing: Run the three scenarios from Chapter 1 with comparable starting conditions. We will record the results and check whether the traffic light changes or route changes cause new queues on other roads.
5. Review and report: Compare the results, improve rules that do not work well, and prepare the final report and demonstration materials.

4. Resource Plan

KATUP is a simulation prototype. We will use our laptops to run SUMO and the Python program. We will try the simulation on the laptops we have. If it runs too slowly, we will use a smaller road network or fewer vehicles during the early stage.

We will use SUMO for the traffic simulation, and Python and TraCI for the three agents. We will use data and graph tools to study the test results. We will use Jira to track our tasks, GitHub to

save code changes, and Google Docs to write the report. We will record the software versions we use so that we can run the simulation again with the same setup.

5. Timeline and Schedule

We plan to work on this project over 14 weeks. Table 2.2 shows the main work and expected result for each week.

Table 2.2. Project Schedule and Weekly Results

Week	Main work	Expected result	Person in charge
1	Choose the problem and write Chapter 1.	Draft of Chapter 1.	All members
2	Write Chapter 2, set up Jira, and prepare GitHub.	Draft of Chapter 2, Jira board, and repository.	Rifqi, with help from all members
3	Write the system needs and rules. Draw the flow between the three agents.	List of needs, first rules, and flow diagram.	All members; Hashif leads the agent rules
4	Build the intersection road network in SUMO Netedit.	Road network file (.net.xml).	Rifqi
5	Add the bottleneck, other routes, and vehicle traffic.	Working simulations of normal and narrow road conditions.	Rifqi
6	Set up TraCI and test whether Python can read data from SUMO.	Python script connected to SUMO through TraCI.	Rifqi & Hashif
7	Build the Observation Agent to read queues and vehicle speeds.	Working queue reading function.	Hashif

8	Connect the basic simulation to the queue reading function and prepare a progress demonstration.	Classroom demonstration of queue data from SUMO.	All members
9	Build the Valve Agent to change the green light time.	Traffic light control through TraCI.	Hashif
10	Build the Route Agent and the digital road sign display.	Route change function and digital sign display.	Fahmi
11	Connect the three agents in one repeated control cycle.	A working KATUP system.	All members
12	Test the three scenarios with comparable starting conditions.	Data on waiting time, travel time, queue length, queues reaching the intersection, vehicles passing through, and conditions on the other route.	All members; Fahmi leads data collection
13	Compare results and change the rules if problems are found.	Comparison tables and graphs, with notes on rule changes.	Fahmi & Hashif
14	Finish the report, demonstration video, and presentation slides.	Final report and presentation materials.	All members

6. Implementation of Project Management Tools (Jira)

We plan to use Jira to record our tasks and follow their progress. Each task will have a name, a person in charge, a target week, and an expected result. We plan to use Backlog, To Do, In Progress, Review/Testing, and Done to show the stage of each task.

A task stays in Backlog while it is still being planned. We move tasks chosen for the current week to To Do, and then to In Progress when work starts. After we finish a result, another member checks it in Review/Testing. We move a task to Done when its result can be run, viewed, or shown during a demonstration.

Table 2.3 shows the tasks we plan to manage in Jira. The task numbers in this table are for our project plan. We will update the task statuses as the work progresses.

Draft ID	Task	Person in charge	Estimate	Draft Status
TASK-01	Set up the Jira board and GitHub repository.	Rifqi	2 days	Done
TASK-02	Build the SUMO road model, including the intersection, bottleneck, and other route.	Rifqi	5 days	In Progress
TASK-03	Write the user needs, first rules for the three agents, and flow diagram.	Hashif	4 days	To Do
TASK-04	Read queue length and vehicle speed from SUMO.	Hashif	5 days	Backlog
TASK-05	Change the traffic light timing and	Hashif	4 days	Backlog

	check the queue before the light.			
TASK-06	Choose another route and move some vehicles to it.	Fahmi	3 days	Backlog
TASK-07	Show route information on the digital road sign in the simulation.	Fahmi	3 days	Backlog
TASK-08	Prepare data recording and test results for the three scenarios using the measures in Chapter 1.	Fahmi, with help from all members	3 days	Backlog
TASK-09	Connect the three agents and test how they share data.	All members	2 days	Backlog

For TASK-08, we will record waiting time, travel time, queue length on both sides of the intersection, how often a queue reaches the intersection, the number of vehicles passing the bottleneck, and conditions on the other route.

7. Risk Management and Mitigation Plan

We also list possible problems during development and how we plan to handle them in Table 2.4.

Table 2.4. Project Risks and Responses

Risk	Level	Our response	Person in charge

Members use different versions of SUMO or Python.	Medium	We will record the versions used by all members and test whether the simulation runs on another member's laptop.	Rifqi
A new queue forms before the traffic light toward the bottleneck.	High	We will check queues on both sides of the intersection. The Valve Agent will stop reducing the green time if the queue before the light gets close to the road's capacity. We will choose the traffic light limits after testing them.	Hashif
The other route becomes crowded.	High	We will move vehicles in stages and check the other route. We will reduce or stop route changes if its queue continues to grow.	Fahmi
The TraCI connection stops during a long simulation.	Medium	We will record the error. The program will not change traffic lights or routes using data that is no longer available. We will test how to restart the simulation or reconnect the program.	Rifqi
Members do not have enough time to finish all tasks.	Medium	We will divide the work into small tasks in Jira and check progress every week. We will work on the simulation and the main functions of the three agents first.	All members

Chapter 3. System Design and UML Diagrams

In this chapter, we explain the system design for the KATUP prototype. We present the high-level architecture (Helicopter View), six Unified Modeling Language (UML) diagrams, and two user interface designs. This design directly follows the goals from Chapter 1 and the project plan from Chapter 2. It shows how the three agents work together to prevent traffic queues from blocking the intersection.

1. Helicopter View of the System Design

The KATUP system is built as a closed-loop control system. It links the SUMO traffic simulator with a Python multi-agent program. Figure 3.1 shows the high-level architecture of KATUP across four main layers.

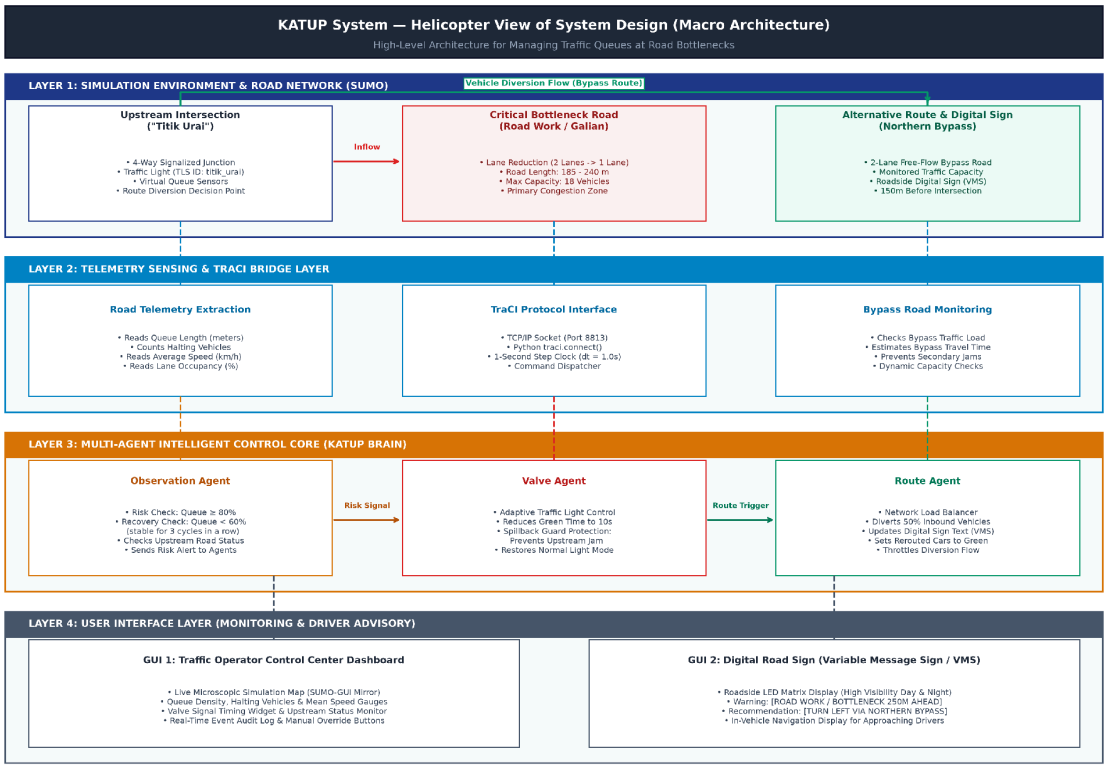


Figure 3.1. Helicopter View of the KATUP Multi-Tier System Design

The system is divided into four functional layers:

- **Layer 1 - Simulation Environment (SUMO Domain):** This layer models the physical road network in SUMO. It contains the 4-way intersection (the control point), the main incoming road (edge_W2J), the bottleneck section where two lanes reduce to one lane (edge_pre_bottle), and the northern detour route. Virtual loop detectors placed on the road measure vehicle counts and average speeds.
- **Layer 2 - Protocol Interface (TraCI TCP/IP Bridge):** This layer connects SUMO to Python using a TCP/IP network socket. Every simulation second, it sends road measurements from SUMO to Python, and sends control commands (traffic light timing and vehicle route changes) back to SUMO.
- **Layer 3 - Multi-Agent Intelligent Core (KATUP Brain):** This layer contains the three Python agents. The Observation Agent checks queue levels. The Valve Agent adjusts the green light duration. The Route Agent moves vehicles to the alternate route when needed.
- **Layer 4 - User Interface Layer (HMI and Monitoring):** This layer provides two visual screens. The first screen is the Traffic Operator Dashboard for traffic managers. The second screen is the Digital Road Sign (Variable Message Sign / VMS) placed before the intersection to advise drivers.

Closed-Loop Control Cycle and System Rules

KATUP works in a continuous cycle: Sense -> Plan -> Act -> Sense. In each step, the Observation Agent reads road conditions. If a queue forms, the Valve Agent and Route Agent take action together. To make sure the system operates smoothly, we set three clear engineering rules:

1. **Bottleneck Risk Detection:** The Observation Agent checks the bottleneck road. If the queue reaches 60% of road capacity (11 out of 18 vehicles) or if vehicle speed drops below 15 km/h with 40% queue, KATUP starts bottleneck mode.
2. **Preventing Quick Mode Switching (Hysteresis Rule):** Traffic control should not switch back and forth too quickly. To keep traffic lights stable, KATUP returns to normal mode only when the queue stays below 30% and speed stays above 25 km/h for at least two cycles in a row.
3. **Protecting Upstream Roads and Balanced Detours:** If vehicles waiting before the traffic light build up too much, the Valve Agent stops reducing green time and keeps at

least 10 seconds of green. Meanwhile, the Route Agent diverts exactly 50% of oncoming vehicles to the northern bypass so the detour road does not become crowded.

2. Unified Modeling Language (UML) Diagrams

To design the software clearly before coding, we created six standard UML diagrams. These diagrams explain the users, classes, message sequences, workflow, states, and system deployment.

2.1. Use Case Diagram

The Use Case Diagram defines what users can do with the system. It connects the user stories from Chapter 1 to system functions. As shown in Figure 3.2, the system has two main users: the Driver and the Traffic Operator, along with the SUMO Simulation Engine.

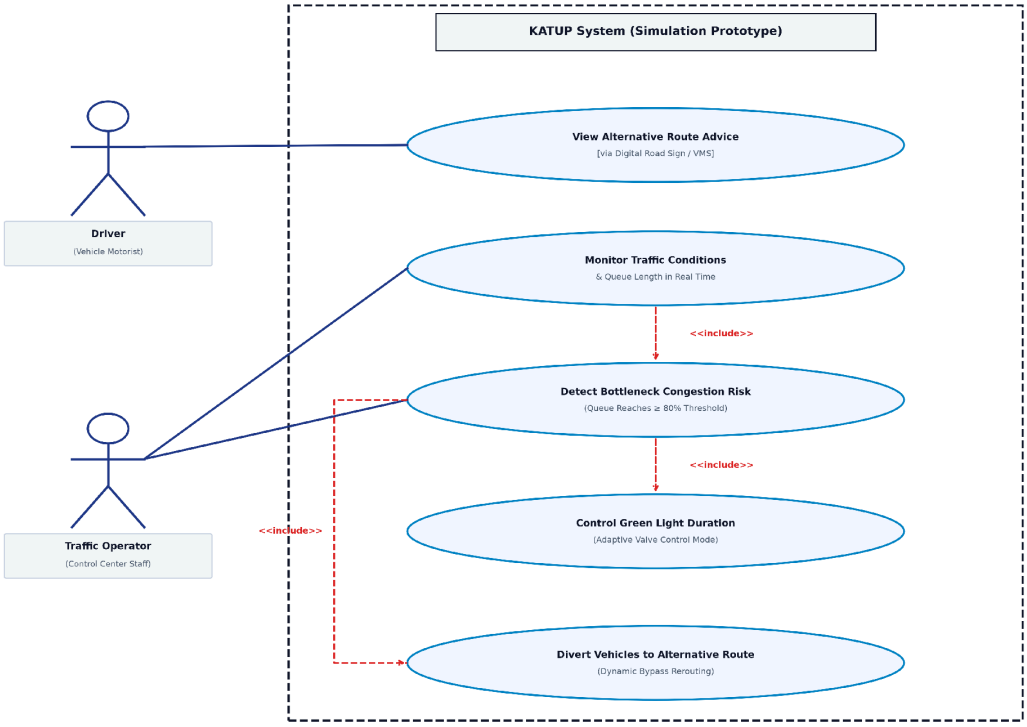


Figure 3.2. Use Case Diagram of the KATUP System

Table 3.1. Use Case Actors and Functional Descriptions

Actor / Use Case	Relationship Type	Functional Description
Driver (Motorist)	Primary External Actor	Drives approaching the intersection. Reads the digital road sign (VMS) and follows route advice to avoid traffic.
Traffic Operator	Primary Human Actor	Monitors traffic queues on the dashboard, views agent actions, and can trigger manual override if needed.
Detect Bottleneck Risk	<<include>> from Monitoring	Reads vehicle count and speed from the bottleneck road. Triggers active control when queue reaches 60%.
Adaptive Valve Control	<<include>> from Risk Detection	Reduces green light duration to 10 seconds at the intersection to stop too many cars from entering the bottleneck.
Dynamic Route Guidance	<<include>> from Risk Detection	Shows detour advisory on the digital road sign and diverts 50% of oncoming vehicles to the northern bypass route.

Note on <<include>> relationships: In Figure 3.2, 'Adaptive Valve Control' and 'Dynamic Route Guidance' use the <<include>> relationship because KATUP runs both actions together whenever a bottleneck risk is detected.

2.2. Class Diagram

The Class Diagram shows the object-oriented structure of our Python software. Each class has a clear role and contains specific variables and functions, as shown in Figure 3.3.

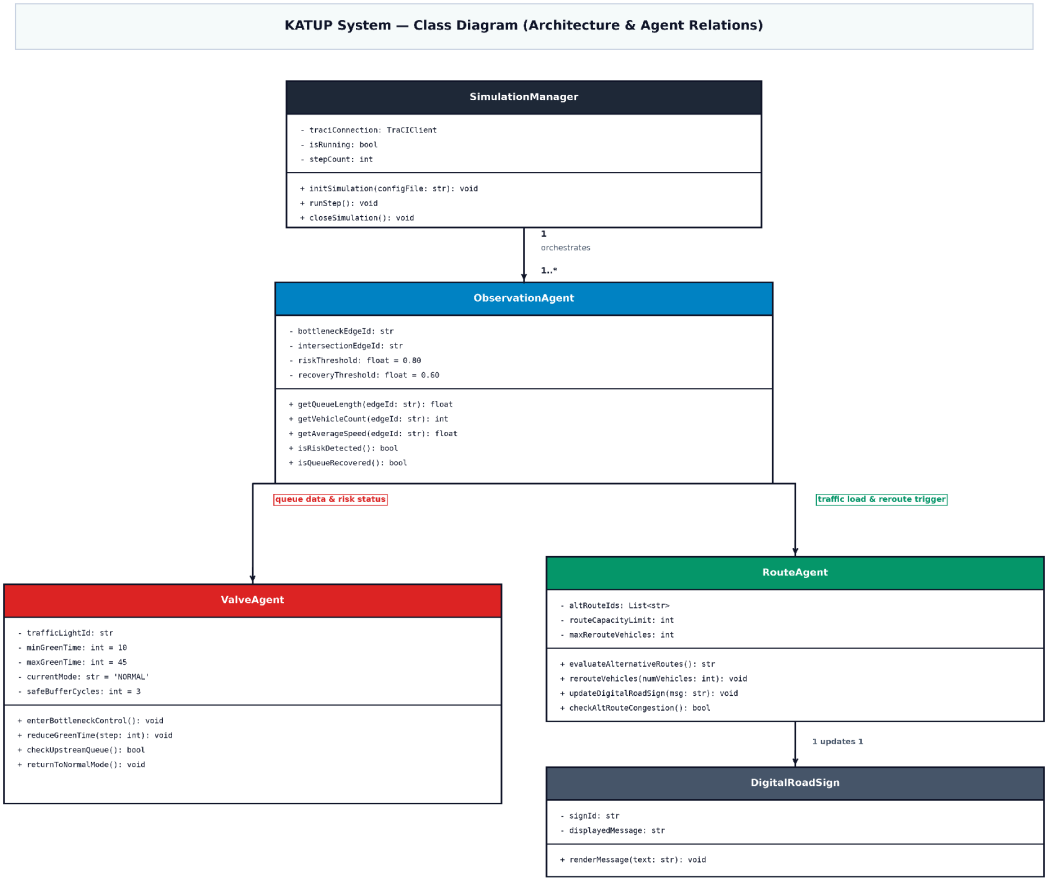


Figure 3.3. Class Diagram of the KATUP Multi-Agent Architecture

Table 3.2. Description of Core Classes and Methods

Class Name	**Key Attributes**	**Key Methods and Responsibilities**
SimulationManager	traciConnection, isRunning, timeStep	run_simulation(), initializeTraCI() Manages the 1800-step simulation loop and controls the connection to SUMO.
ObservationAgent	edge_id, consecutive_normal_count, max_capacity	evaluate() Reads vehicle count and speed from SUMO, calculates queue percentage, and detects risk.

ValveAgent	tls_id, mode ('NORMAL' / 'KATUP_ACTIVE')	apply_control() Checks current traffic light phase and shortens green light to 10 seconds during active mode.
RouteAgent	inbound_edge, bypass_route, reroute_counter	guide_traffic() Finds oncoming vehicles, diverts 50% to the bypass route, and changes their color to green.
DigitalRoadSign	sign_id, current_message, is_warning_active	renderMessage(), setDisplayState() Controls the text shown on the roadside electronic display for drivers.

2.3. Sequence Diagram

The Sequence Diagram in Figure 3.4 shows the order of messages between objects during each simulation time step (dt = 1.0 second).

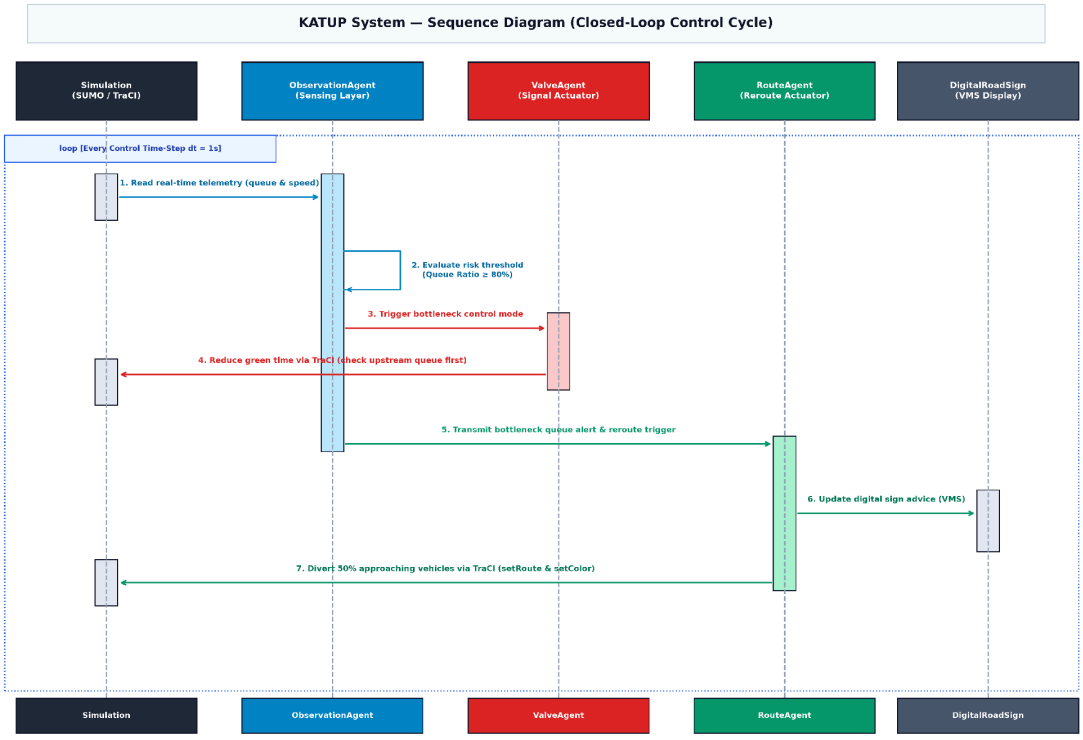


Figure 3.4. Sequence Diagram of the Closed-Loop Control Cycle

Table 3.3. Step-by-Step Message Flow in the Closed-Loop Cycle

Step	Message Flow (Sender -> Receiver)	Purpose and Logic
1	SimulationManager -> SUMO (via TraCI)	Calls traci.simulationStep() to advance the simulation by 1 second.
2	ObservationAgent -> SUMO	Reads vehicle count, halting cars, and average speed on the bottleneck road.
3	ObservationAgent (internal logic)	Calculates queue percentage. Flags is_high_risk = True if queue >= 60% or speed <= 15 km/h.
4	ValveAgent -> SUMO Traffic Light	If risk is high and green light has run for 10 seconds, commands the signal to switch to yellow.
5	RouteAgent -> SUMO Vehicles	Finds oncoming vehicles on the approach road and reroutes 50% to the bypass road.
6	RouteAgent -> SUMO GUI Display	Sets diverted vehicle color to green so operators can visually track rerouted traffic.

2.4. Activity Diagram

The Activity Diagram in Figure 3.5 models the complete decision flow of the KATUP control program across three swimlanes: SUMO Simulator, Observation Agent, and Valve & Route Agents.

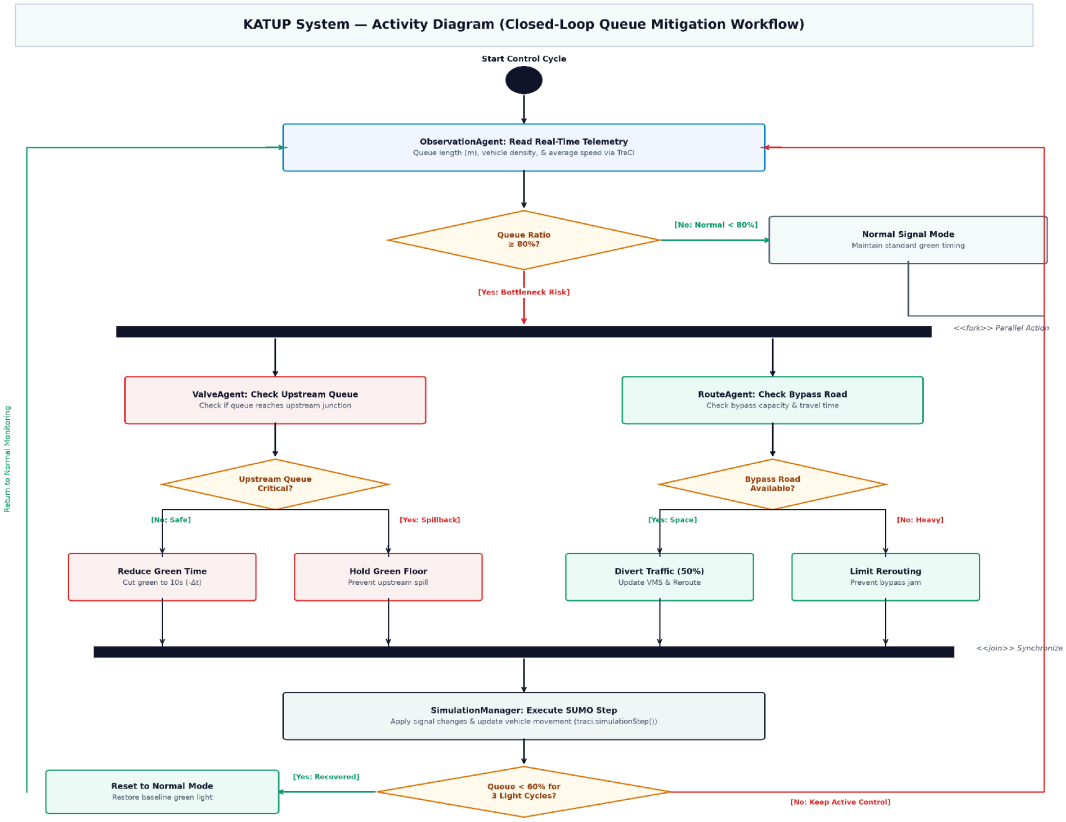


Figure 3.5. Activity Diagram of the Intelligent Control Algorithm

Table 3.4. Activity Workflow Stages and Guard Conditions

Workflow Stage	**Swimlane / Responsible Component**	**Stage Description**
1. Read Road Telemetry	SUMO & Observation Agent	Simulates 1 second and collects vehicle counts and speeds from road detectors.
2. Risk Decision Branch	Observation Agent	Checks if queue ratio >= 60%. If yes, moves to parallel action fork. If no, checks recovery.
3. Parallel Action Fork	Valve Agent & Route Agent	Runs two actions at once: Valve Agent limits green light to 10s; Route Agent diverts 50% of vehicles.

4. Parallel Join & Reset	Valve Agent & Route Agent	Both actions complete. The consecutive normal counter is reset to 0.
5. Recovery Check	Observation Agent	If queue <= 30% and speed >= 25 km/h for 2 cycles in a row, the system returns to Normal Mode.

2.5. State Machine Diagram

The State Machine Diagram in Figure 3.6 shows the lifecycle and modes of the Valve Agent during operation.

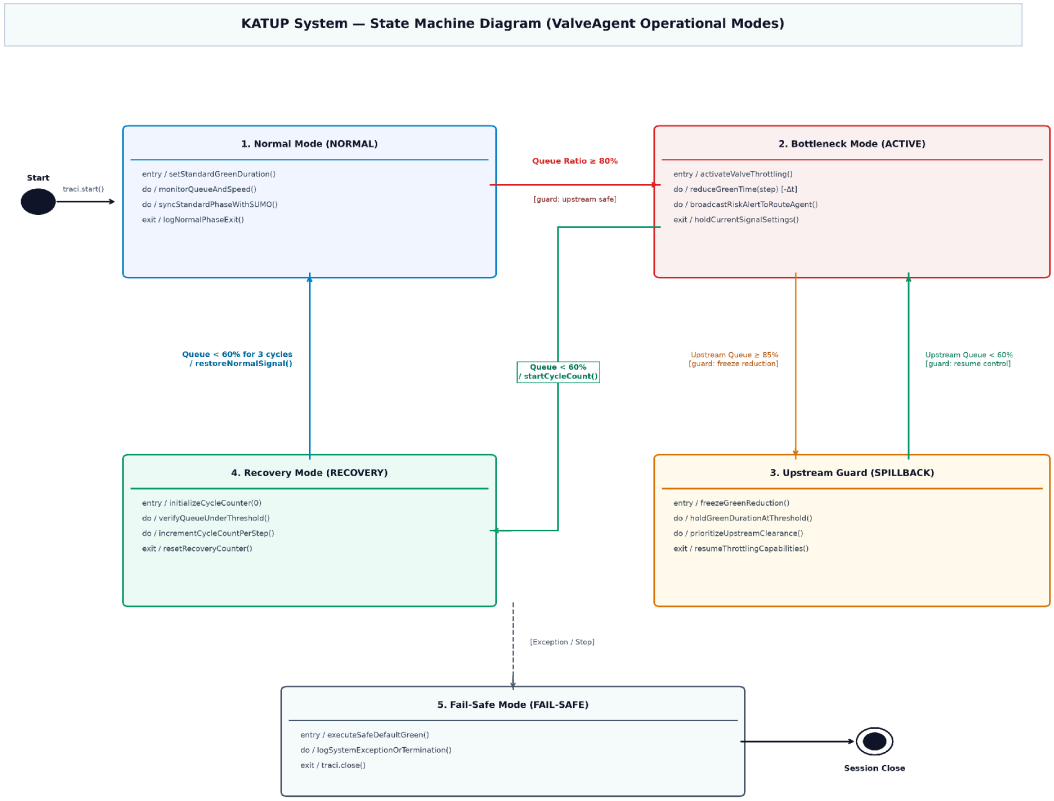


Figure 3.6. State Machine Diagram of the Valve Agent

Table 3.5. Valve Agent States, Internal Actions, and Transitions

State Name	**Internal Actions**	**Transition Triggers and Conditions**

INITIALIZING	Load road network and connect TraCI socket on port 8813.	Goes to NORMAL_MONITORING when connection is ready.
NORMAL_MONITORING	Traffic light runs default timing. Digital sign shows clear road.	Transitions to KATUP_ACTIVE if queue >= 60% or speed <= 15 km/h.
KATUP_ACTIVE	Limits green time to 10 seconds. Reroutes 50% of cars to bypass road.	Transitions to RECOVERY_BUFFERING if queue <= 30% and speed >= 25 km/h.
RECOVERY_BUFFERING	Maintains restricted green time while verifying queue does not return.	Returns to NORMAL if clean for 2 cycles; returns to ACTIVE if queue spikes.
FAILSAFE_RELIEF	Enforces 10-second minimum green light to prevent upstream gridlock.	Triggered if queue before intersection reaches 85% capacity.

2.6. Component and Deployment Diagram

The Component and Deployment Diagram in Figure 3.7 shows how the software parts are organized and where they run on the computer.

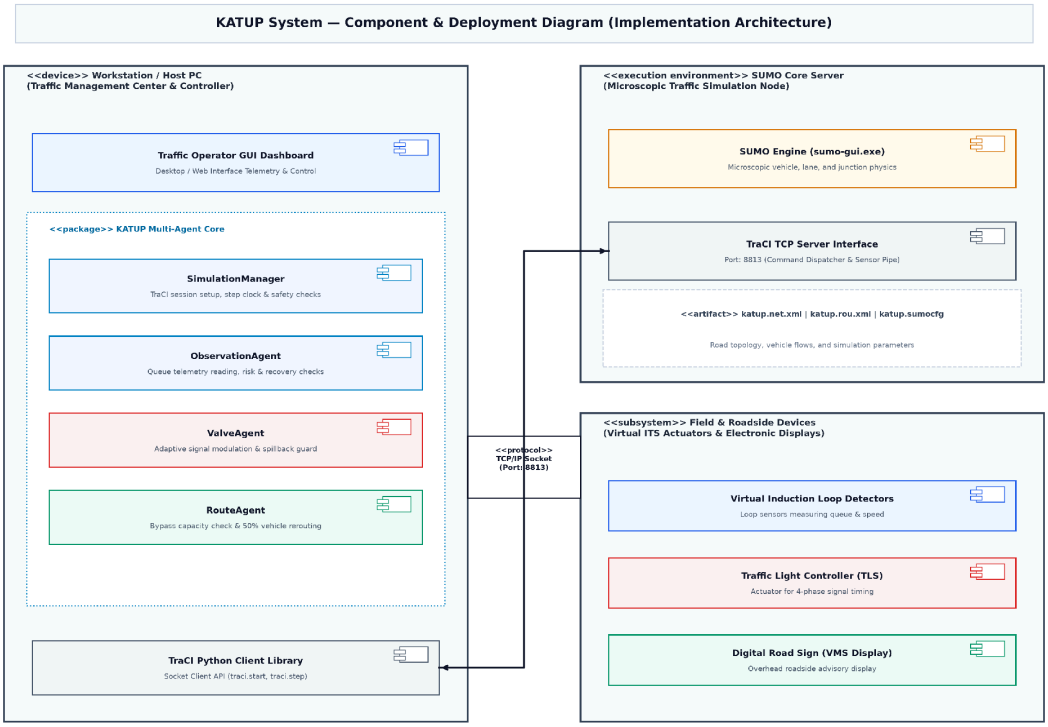


Figure 3.7. Component and Deployment Diagram of the KATUP System

Table 3.6. Deployment Nodes, Software Artifacts, and Protocols

Deployment Node	**Software Components**	**Role and Communication Protocol**
Simulation Workstation	SUMO 1.20+ Engine (sumo-gui.exe) Network (.net.xml) & Routes (.rou.xml)	Runs microscopic traffic simulation. Exposes TraCI TCP socket on port 8813.
KATUP Control Node	SimulationManager, ObservationAgent, ValveAgent, RouteAgent (Python 3.12)	Runs multi-agent control logic. Communicates with SUMO via TraCI library.
Traffic Operator Console	KATUP Dashboard GUI Application (Tkinter / Web Interface)	Displays real-time map, queue gauges, and event log for traffic operators.
Roadside Sign Node	DigitalRoadSign VMS Module	Displays visual route advisories to drivers approaching the intersection.

3. Graphical User Interface (GUI) Specification

In this simulation prototype, we use SUMO-GUI as our graphical user interface. SUMO-GUI displays the roads, vehicles, traffic lights, and queues in real time. We also use the Python console terminal to show live data from the simulation. Together, these two screens help the traffic operator monitor road conditions and check whether the three agents work properly.

3.1. Traffic Operator Simulation Interface (SUMO-GUI)

Figure 3.8 shows the full view of the road network in SUMO-GUI. From this view, the traffic operator can watch the whole corridor: the four-way intersection (J1), the bottleneck road on the right, and the northern bypass road at the top.



Figure 3.8. SUMO-GUI Traffic Operator Simulation Interface

Table 3.7. SUMO-GUI Visual Elements and Operator Functions

Interface Element	Visual Display in SUMO-GUI	Function for Operator

Road Network View	2D map showing the intersection, bottleneck road, and bypass road	Helps the operator see vehicle movement and traffic flow in real time.
Traffic Light Display	Traffic signal at the intersection showing red, yellow, and green lights	Shows when the Valve Agent changes or reduces the green light time.
Bottleneck Area	Vehicles lining up before the road narrows from two lanes to one lane	Shows where the queue forms and how long it grows toward the intersection
Northern Bypass Route	The alternative road at the top of the map	Shows the extra road used to reduce traffic on the main road.
Green Vehicle Flow	Vehicles displayed in bright green color	Shows vehicles that have been redirected by the Route Agent to the bypass road.
Console Output Window	Terminal text showing step time, vehicle count, speed, and KATUP mode	Gives exact numbers for queue density, speed, and agent status every 15 seconds.

3.2. Vehicle Display and Route Guidance (Virtual VMS)

To meet the driver user story from Chapter 1 ("As a driver, I want to see information about another route on a digital road sign."), our system guides some vehicles to take the bypass road before they enter the narrow road.

In SUMO-GUI, we show this guidance by changing the vehicle color. Figure 3.9 shows a closer view of the intersection. Vehicles approaching the intersection from the west are shown in two colors: normal vehicles remain yellow, while vehicles that receive route advice change to bright green and turn left into the bypass road.



Figure 3.9. Vehicle Guidance and Route Choice at the Intersection

Table 3.8. Digital Road Sign Display Elements and Driver Interaction

Visual Element	How It Looks in SUMO-GUI	Meaning and Action
Green Vehicles	Vehicles colored bright green	Vehicles that take the bypass route to avoid the queue.
Yellow Vehicles	Vehicles colored yellow	Vehicles that stay on the main route toward the bottleneck.
Decision Point	Road section 150 meters	The area where the Route Agent chooses cars to turn left before the light.
50% Reroute Split	Green and yellow cars alternating in the queue	Shows that the Route Agent only moves half of the cars so the bypass does not jam.
Console VMS Status	Text showing "[VMS: AKTIF (Bypass Hijau)]"	Simulates the digital road sign message telling drivers to use the bypass.

3.3. Connection Between Simulation Display and Python Code

To make sure our visual display matches the actual program, Table 3.9 shows how the elements in SUMO-GUI connect to the Python code and TraCI functions.

Table 3.9. GUI Component to Agent Code Traceability

Visual Element	**Related Python Class and Method**	**What the Code Does**
Queue and Vehicle Count	AgenPengamat.evaluate()	Reads vehicle number and speed on the road using traci.edge functions.
Traffic Light Changes	AgenKatup.apply_control()	Reduces green light duration using traci.trafficlight.setPhase().
Green Car Colors	AgenRute.guide_traffic()	Changes vehicle color to green using traci.vehicle.setColor().
Bypass Route Change	AgenRute.guide_traffic()	Changes vehicle path to the bypass using traci.vehicle.setRoute().
Console Log Text	run_simulation()	Prints step number, vehicle count, speed, and KATUP mode every 15 seconds.

4. Design Alignment and Summary

The system design described in this chapter connects directly to our project goals from Chapter 1 and Chapter 2:

- **Alignment with Chapter 1 (User Stories):** The driver story is met because drivers get route advice, which we show as green cars taking the bypass road (Figure 3.9). The traffic operator story is met because operators can watch the queues, light timing, and agent actions in SUMO-GUI and the console (Figure 3.8).
- **Alignment with Chapter 2 (Tasks and Schedule):** TASK-03 is completed through our UML diagrams and rules. TASK-04, TASK-05, TASK-06, and TASK-07 are connected directly to our simulation files (katup.net.xml, katup.rou.xml) and Python script (katup_controller.py).

With this design completed, our group is ready to run the three test scenarios and collect experimental data in Chapter 4.

Chapter 4. System Implementation, Coding Progress, and User Memorandum of Understanding

1. Progressive Coding Stages and Architecture Evolution

The software implementation was engineered across five progressive stages in the repository to ensure modularity, reproducibility, and systematic verification against the UML designs established in Chapter 3. Table 4.1 summarizes each implementation stage and its generated software deliverables.

Table 4.1. Progressive Coding Stages, Technical Milestones, and Architectural Deliverables

Stage / Folder	Focus & Objectives	Generated Artifacts	Key Implementation Deliverables
Stage 1: 01_simpang_dasar	Base Signalized Junction	tahap1.nod.xml tahap1.edg.xml tahap1.net.xml	Built a 4-way signalized intersection (J1) with 2-lane arterial corridors; compiled via netconvert.
Stage 2: 02_galian_dan_bypass	Bottleneck & Bypass Topology	tahap2.nod.xml tahap2.edg.xml tahap2.con.xml	Created a construction bottleneck (lane reduction 2 to 1) and a dedicated 2-lane Northern Bypass route.
Stage 3: 03_skenario_arus_kendaraan	Congestion Demand Injection	tahap3.rou.xml tahap3.sumocfg	Calibrated peak vehicle flows (1800+ veh/hr) to induce queue spillback toward junction J1.
Stage 4: 04_multi_agent_traci	TraCI Multi-Agent Integration	katup_controller.py katup.rou.xml	Implemented Python TraCI socket controller with AgenPengamat, AgenKatup, and AgenRute closed loops.
Stage 5: 05_visual_urban_enhancements	Realistic Urban Visualization	katup.poly.xml katup.view.xml build_network.py	Added 2D vehicle models with active headlights/brake lights, river canal, city park, buildings, and VMS.

1.1. Stage 1: Base Signalized Intersection (01_simpang_dasar)

In Stage 1, node coordinates were established around central junction J1 at (0, 0) with two-lane arterial approaches. Using SUMO netconvert, lane connections, turning rights, and fixed-time signal cycles (30s green, 3s yellow, 2s all-red) were validated to form the baseline network.

1.2. Stage 2: Road Bottleneck and Northern Bypass (02_galian_dan_bypass)

Stage 2 modeled the physical congestion cause: an urban excavation zone located 185 m east of J1 on edge_pre_bottle, narrowing the road from two lanes to a single lane. To accommodate diverted vehicles, a four-segment northern bypass corridor (edge_W2J, edge_J2N, edge_alt_1, edge_alt_2, edge_post_merge) was engineered with explicit left-turn connections.

1.3. Stage 3: Congestion Demand Generation (03_skenario_arus_kendaraan)

Arterial traffic flows were injected at 1800 vehicles per hour on the inbound western approach, exceeding the single-lane bottleneck capacity (1200 veh/hr). Uncontrolled simulations confirmed that queues reached the maximum link storage (18 vehicles) within 240 seconds, spilling into intersection J1 and blocking all cross-traffic movements.

1.4. Stage 4: TraCI Multi-Agent System Integration (04_multi_agent_traci)

The Python controller script katup_controller.py was developed to link external multi-agent logic with SUMO via TraCI. Three cooperating agents were implemented: AgenPengamat (queue sensing), AgenKatup (adaptive green-light throttling), and AgenRute (proportional vehicle rerouting and dynamic neon-green coloring).

1.5. Stage 5: Realistic Urban Visual Enhancements (05_visual_urban_enhancements)

To provide stakeholder-grade demonstration clarity, realistic 2D vehicle rendering was integrated with active headlights, turn indicators, and braking lights. Environmental polygons were added: an urban river canal with bridges, a central city park with a decorative lake, commercial 3D buildings, pedestrian sidewalks, and construction safety barricades.

2. Collaborative Git Workflow and Version Control Progression

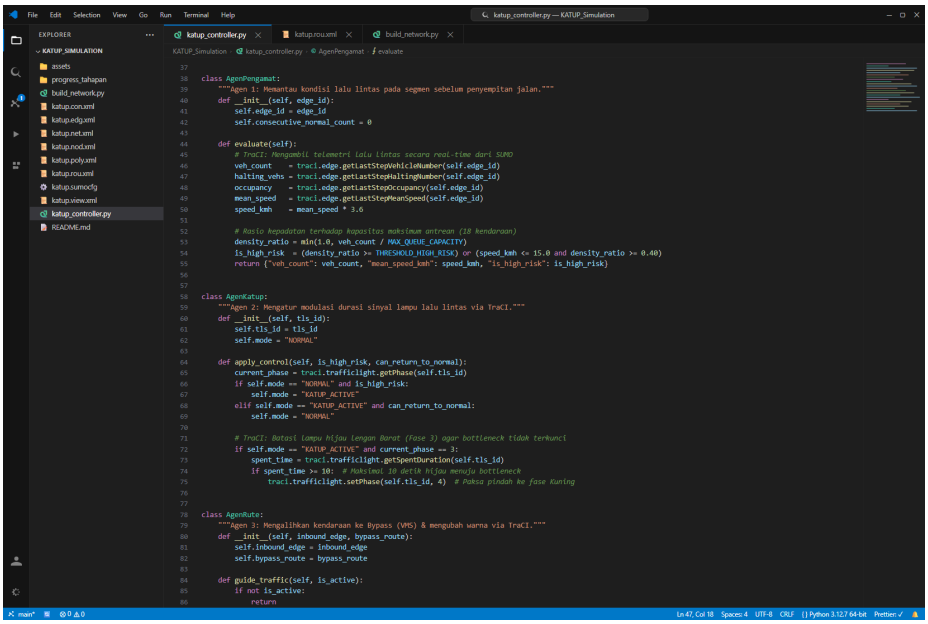
Version control was managed via Git using Conventional Commits to maintain clear task traceability to Jira. Table 4.2 presents the chronological Git commits representing the authentic implementation progression.

Table 4.2. Sequential Git Commit Progression and Software Deliverables

Commit Hash	Author & Team Role	Commit Message / Conventional Title	Modified Files	Verified Functionality
9304710	Muhamad Rifqi (Lead)	feat(net): Tahap 1 - Pemodelan Jaringan Dasar Simpang 4 Bersinyal	tahap1.nod/edg/net.xml	Base 4-way signalized intersection verified.
e78cbe5	Muhamad Rifqi (Lead)	feat(network): Tahap 2 - Pemodelan Bottleneck Galian dan Bypass	tahap2.nod/edg/con.xml	Bottleneck & 2-lane northern bypass verified.
fa62dae	Muhamad Rifqi (Lead)	feat(demand): Tahap 3 - Skenario Arus Beban Puncak	tahap3.rou.xml, sumocfg	Peak demand & gridlock spillback verified.
a279689	Hashif & Fahmi	feat(controller): Tahap 4 - Integrasi Sistem Multi-Agen TraCI	katup_controller.py	3 agents linked via TraCI TCP port 8813.
9f2ab1c	Muhamad Rifqi (Lead)	feat(visual): Tahap 5 - Visual Koridor Multi-Simpang Realistis	katup.poly/view.xml	2D cars, canal, park, and buildings verified.
18857da	Group 03	docs(report): Bab 4 Progres Implementasi, MoU Dishub, dan RAB	make_chapter4_doc.py	Academic report, MoU, and RAB completed.

3. TraCI Multi-Agent Source Code Implementation and Walkthrough

The multi-agent control logic resides in katup_controller.py, communicating with SUMO via TraCI. Figure 4.1 and Figure 4.2 illustrate the Visual Studio Code implementation of the three agents and the main execution loop.



```python
class AgenPengamat:
    """Agen 1: Pemantau kondisi lalu lintas pada segmen sebelum penyempitan jalan."""
    def __init__(self, edge_id):
        self.edge_id = edge_id
        self.consecutive_normal_count = 0

    def evaluate(self):
        # TraCI: Mengambil telemetri lalu lintas secara real-time dari SUMO
        veh_count    = traci.edge.getLastStepVehicleNumber(self.edge_id)
        halting_vehs = traci.edge.getLastStepHaltingNumber(self.edge_id)
        occupancy    = traci.edge.getLastStepOccupancy(self.edge_id)
        mean_speed   = traci.edge.getLastStepMeanSpeed(self.edge_id)
        speed_kmh    = mean_speed * 3.6

        # Rasio kepadatan terhadap kapasitas maksimum antrean (18 kendaraan)
        density_ratio = min(1.0, veh_count / MAX_QUEUE_CAPACITY)
        is_high_risk  = (density_ratio >= THRESHOLD_HIGH_RISK) or (speed_kmh <= 15.0 and density_ratio >= 0.40)
        return {"veh_count": veh_count, "mean_speed_kmh": speed_kmh, "is_high_risk": is_high_risk}


class AgenKatup:
    """Agen 2: Mengatur modulasi durasi sinyal lampu lalu lintas via TraCI."""
    def __init__(self, tls_id):
        self.tls_id = tls_id
        self.mode = "NORMAL"

    def apply_control(self, is_high_risk, can_return_to_normal):
        current_phase = traci.trafficlight.getPhase(self.tls_id)
        if self.mode == "NORMAL" and is_high_risk:
            self.mode = "KATUP_ACTIVE"
        elif self.mode == "KATUP_ACTIVE" and can_return_to_normal:
            self.mode = "NORMAL"

        # TraCI: Batasi lampu hijau lengan Barat (Fase 3) agar bottleneck tidak terkunci
        if self.mode == "KATUP_ACTIVE" and current_phase == 3:
            spent_time = traci.trafficlight.getSpentDuration(self.tls_id)
            if spent_time >= 10:  # Maksimal 10 detik hijau menuju bottleneck
                traci.trafficlight.setPhase(self.tls_id, 4)  # Paksa pindah ke fase Kuning


class AgenRute:
    """Agen 3: Mengalihkan kendaraan ke Bypass (VMS) & mengubah warna via TraCI."""
    def __init__(self, inbound_edge, bypass_route):
        self.inbound_edge = inbound_edge
        self.bypass_route = bypass_route

    def guide_traffic(self, is_active):
        if not is_active:
            return
```

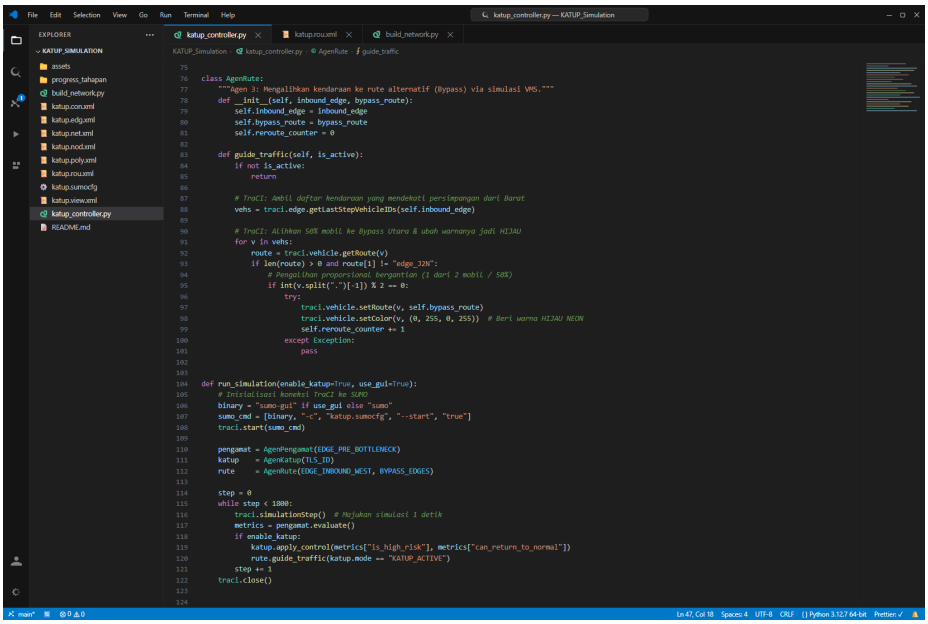
Figure 4.1. Visual Studio Code Implementation of AgenPengamat and AgenKatup Logic

3.1. Observation Agent (AgenPengamat) Implementation

AgenPengamat samples road traffic variables on edge_pre_bottle every simulation second. It queries traci.edge.getLastStepVehicleNumber() and traci.edge.getLastStepHaltingNumber(), computes the density ratio (veh_count / MAX_QUEUE_CAPACITY, where capacity = 18), and evaluates speed in km/h. High risk is triggered when density >= 60% or speed <= 15 km/h with density >= 40%. A two-cycle safe buffer (density <= 30%, speed >= 25 km/h) prevents premature deactivation.

3.2. Valve Agent (AgenKatup) Implementation

When active risk is signaled, AgenKatup monitors signal phase 3 (West green light toward the bottleneck). If elapsed green time reaches 10 seconds (traci.trafficlight.getSpentDuration >= 10), the agent immediately calls traci.trafficlight.setPhase(tls_id, 4), forcing a transition to yellow. This throttles incoming bottleneck traffic by ~66%.



```python
class AgenRute:
    """Agen 3: Mengalihkan kendaraan ke rute alternatif (Bypass) via simulasi VMS."""
    def __init__(self, inbound_edge, bypass_route):
        self.inbound_edge = inbound_edge
        self.bypass_route = bypass_route
        self.reroute_counter = 0

    def guide_traffic(self, is_active):
        if not is_active:
            return

        # TraCI: Ambil daftar kendaraan yang mendekati persimpangan dari Barat
        vehs = traci.edge.getLastStepVehicleIDs(self.inbound_edge)

        # TraCI: Alihkan 50% mobil ke Bypass Utara & ubah warnanya jadi HIJAU
        for v in vehs:
            route = traci.vehicle.getRoute(v)
            if len(route) > 0 and route[1] != "edge_J2N":
                # Pengalihan proporsional bergantian (1 dari 2 mobil / 50%)
                if int(v.split(".")[-1]) % 2 == 0:
                    try:
                        traci.vehicle.setRoute(v, self.bypass_route)
                        traci.vehicle.setColor(v, (0, 255, 0, 255))  # Beri warna HIJAU NEON
                        self.reroute_counter += 1
                    except Exception:
                        pass


def run_simulation(enable_katup=True, use_gui=True):
    # Inisialisasi koneksi TraCI ke SUMO
    binary = "sumo-gui" if use_gui else "sumo"
    sumo_cmd = [binary, "-c", "katup.sumocfg", "--start", "true"]
    traci.start(sumo_cmd)

    pengamat = AgenPengamat(EDGE_PRE_BOTTLENECK)
    katup    = AgenKatup(TLS_ID)
    rute     = AgenRute(EDGE_INBOUND_WEST, BYPASS_EDGES)

    step = 0
    while step < 1800:
        traci.simulationStep()  # Majukan simulasi 1 detik
        metrics = pengamat.evaluate()
        if enable_katup:
            katup.apply_control(metrics["is_high_risk"], metrics["can_return_to_normal"])
            rute.guide_traffic(katup.mode == "KATUP_ACTIVE")
        step += 1
    traci.close()
```

Figure 4.2. Visual Studio Code Implementation of AgenRute Dynamic Bypass Guidance

3.3. Route Agent (AgenRute) Implementation

As shown in Figure 4.2, when KATUP is active, AgenRute identifies approaching vehicles on edge_W2J within 150 m of J1. It applies a 50% alternating diversion split based on vehicle ID parity (int(v.split('.')[-1]) % 2 == 0). Selected vehicles are injected with BYPASS_EDGES via traci.vehicle.setRoute() and painted bright neon green ((0, 255, 0, 255)) using traci.vehicle.setColor() for immediate visual identification.

3.4. Simulation Execution Loop and TraCI Socket Bridge

The simulation loop executes 1800 steps (30 minutes of simulated traffic). Each step calls traci.simulationStep(), updates agent decisions, and prints telemetric status to the console every 15 seconds. Robust exception handling ensures that traci.close() is always executed in a finally block to release socket port 8813.

Table 4.3. Core TraCI API Function Calls, Parameters, and Multi-Agent Mapping

TraCI Method	Component	Key Parameters	Operational Role in KATUP
traci.start()	Simulation Loop	sumo_cmd, --start	Initializes SUMO subprocess and establishes TCP socket bridge.
traci.simulationStep()	Simulation Loop	None	Advances microscopic simulation clock by 1.0 second.

TraCI Method	Component	Key Parameters	Operational Role in KATUP
traci.edge.getLastStepVehicleNumber()	AgenPengamat	edge_id	Samples active vehicles on bottleneck approach to compute density.
traci.edge.getLastStepMeanSpeed()	AgenPengamat	edge_id	Reads average velocity to detect crawling bottleneck conditions.
traci.trafficlight.getSpentDuration()	AgenKatup	tls_id	Checks active green phase duration; triggers throttle if >= 10s.
traci.trafficlight.setPhase()	AgenKatup	tls_id, 4	Forces signal transition from green to yellow clearance phase.
traci.vehicle.setRoute()	AgenRute	veh_id, BYPASS_EDGES	Dynamically updates vehicle path to the northern bypass detour.
traci.vehicle.setColor()	AgenRute	veh_id, (0, 255, 0, 255)	Renders diverted vehicles in neon green for operator visibility.
traci.close()	Simulation Loop	None	Safely terminates socket connection and releases TCP port.

4. Empirical Simulation Results and Practical Implementation Lessons

Simulation runs verified that while the uncontrolled baseline caused total gridlock within 4 minutes, the active KATUP system maintained steady queue containment below 60% and preserved full intersection flow. Figure 4.3 and Figure 4.4 illustrate the simulation output in SUMO-GUI.

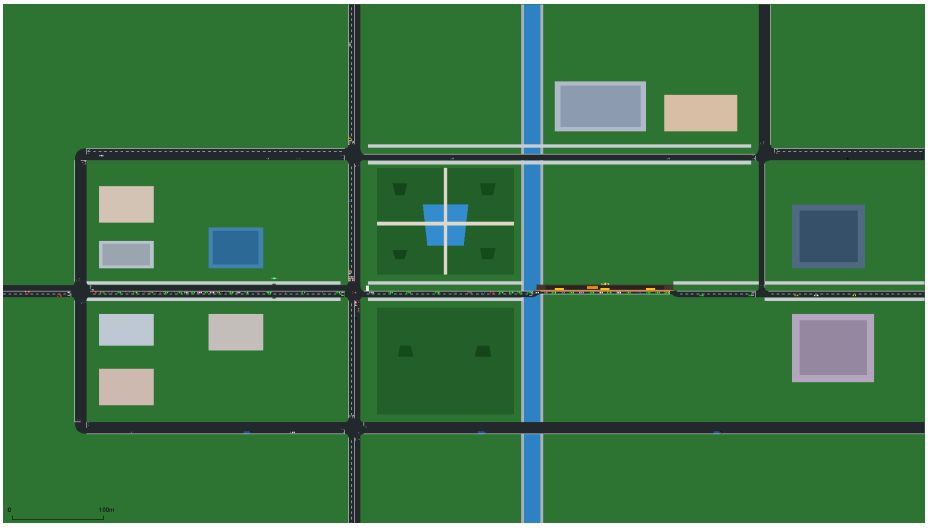


Figure 4.3. SUMO-GUI Full Corridor Simulation View with Active Realistic Urban Environment

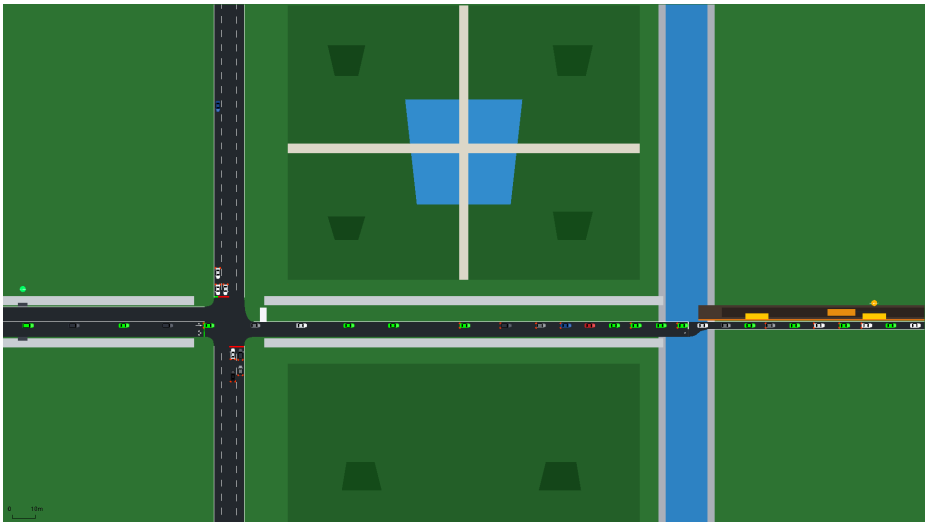


Figure 4.4. Detailed SUMO-GUI Intersection View Showing Neon-Green Rerouted Vehicles and Controlled Traffic Flow

Table 4.4. Observed Empirical Traffic Telemetry and Multi-Agent Mode Transitions

Time Step	Vehicles	Density	Mean Speed	KATUP Mode	VMS Status	Observed Traffic Dynamic
t = 015 s	3 veh	16.7%	42.8 km/h	NORMAL	OFF	Free-flow traffic; no queue buildup.
t = 060 s	7 veh	38.9%	31.2 km/h	NORMAL	OFF	Queue forming before bottleneck; below threshold.
t = 120 s	12 veh	66.7%	14.1 km/h	KATUP_ACTIVE	AKTIF (Bypass Hijau)	Threshold breached; green capped at 10s; 50% detour starts.
t = 240 s	8 veh	44.4%	23.4 km/h	KATUP_ACTIVE	AKTIF (Bypass Hijau)	Queue shrinks; intersection J1 remains 100% clear.
t = 360 s	4 veh	22.2%	34.1 km/h	NORMAL	OFF	Two safe cycles verified; smooth return to Normal mode.

4.1. Practical Implementation Challenges and Lessons Learned

Reflecting the authentic development experience ('seadanya yang sudah dialami saja'), four key technical hurdles were solved:

- **TraCI Socket Persistence:** Closing SUMO-GUI manually raised unhandled FatalTraCIError exceptions, locking port 8813. Enclosing the simulation loop in a try-except-finally block with guaranteed traci.close() resolved the issue.
- **Route Injection Timing:** Calling setRoute() after a vehicle crossed into the intersection caused TraCI exceptions. Scanning vehicles on edge_W2J 150 m upstream ensured reliable route assignment prior to the divergence node.
- **Hysteresis Stabilization:** Single-step thresholding led to rapid mode switching ('hunting'). Introducing a two-cycle hysteresis confirmation buffer (RECOVERY_CYCLES_NEEDED = 2) stabilized signal transitions.
- **Visual Quality for Stakeholders:** Default SUMO geometric dots were unconvincing for presentations. Creating custom 2D vehicle models and realistic urban polygons substantially improved visual clarity and demonstration impact.

5. Memorandum of Understanding (MoU) with Dinas Perhubungan Provinsi DKI Jakarta

Under the academic guidance of Prof. Dr. Ir. Riri Fitri Sari, M.M., M.Sc., Group 03 formulated a formal Memorandum of Understanding (Nota Kesepahaman) with municipal partner Dinas Perhubungan Provinsi DKI Jakarta. The MoU spans fourteen articles (Pasal 1 s.d. 14) balancing student innovation with municipal safety and data governance.

Table 4.5. Structure, Legal Substance, and Engineering Implications of the KATUP - Dishub DKI Jakarta MoU

MoU Article & Title	Legal & Operational Substance	Direct Implications for KATUP Development
Pasal 1: Maksud dan Tujuan	Cooperative exploration to evaluate the KATUP prototype for Jakarta traffic management.	Acknowledges KATUP as an academic research prototype by FTUI students.
Pasal 2: Ruang Lingkup Kerja Sama	7 scopes: demonstration, bottleneck testing, multi-agent evaluation, workshops, and pilot study.	Validates the 3-agent architecture against real Jakarta congestion challenges.
Pasal 3: Bentuk Pelaksanaan	Technical presentations, workshop discussions, and joint simulation evaluations.	Authorizes structured technical sharing sessions with Dishub traffic engineers.

MoU Article & Title	Legal & Operational Substance	Direct Implications for KATUP Development
Pasal 4 & 5: Hak dan Kewajiban	FTUI presents objective test data; Dishub provides operational feedback without financial liability.	Protects both parties by establishing transparent expectations during exploratory phase.
Pasal 6: Data dan Kerahasiaan	Mandates data protection in accordance with UU Perlindungan Data Pribadi No. 27/2022.	Ensures any real municipal traffic telemetry is used strictly for academic research.
Pasal 7: Kekayaan Intelektual (HKI)	Pre-existing software and algorithms remain the exclusive IP of originating party (FTUI Group 03).	Secures student copyright over Python code, XML schemas, and agent algorithms.
Pasal 8 & 9: Pembiayaan & Keselamatan	Exploration is self-funded; strictly prohibits direct control of public signals without field safety clearance.	Enforces laboratory simulation boundaries, preventing premature field deployment.
Pasal 10-14: Jangka Waktu & Penutup	6-month validity period, amicable dispute resolution through deliberation (musyawarah).	Provides a structured 6-month operational window for project evaluation.

5.1. Safety Boundaries, IP Rights (HKI), and Academic Safeguards

Pasal 7 protects student copyright over the TraCI codebase and algorithms. Pasal 8 and 9 strictly classify KATUP as a simulation prototype, prohibiting any live physical connection to municipal ATCS signal cabinets without certified fail-safe hardware relays and formal field safety approvals. This boundary safeguards municipal traffic while fostering unhindered academic simulation research.

6. Financial Planning: Rencana Anggaran Biaya (RAB) for Pilot Deployment

Pursuant to Pasal 8 of the MoU, the team structured a Project Budget Plan (Rencana Anggaran Biaya - RAB) to pilot KATUP across one signalized Jakarta corridor over three months. Table 4.6 details the itemized expenditures.

Table 4.6. Summary of KATUP Pilot Deployment Budget Plan (RAB Proyek 2026)

No	Expenditure Category	Sub-Items & Technical Scope	Allocated Cost (IDR)
1.0	Pengembangan & Refinement Perangkat Lunak	Multi-Agent TraCI optimization, edge AI CCTV analytics, and web dashboard.	Rp 48.000.000
2.0	Pengadaan Perangkat Keras (Hardware & Edge)	Industrial Edge Controller (Jetson/Pi 5), 4G/5G cellular gateway, and mini-VMS.	Rp 46.500.000
3.0	Pengujian Lapangan & Integrasi Sistem ATCS	API middleware integration with Jakarta ATCS and peak-hour calibration.	Rp 34.000.000
4.0	Operasional, Pelatihan Operator & Diseminasi	Dishub operator training workshops, documentation, and symposium publication.	Rp 26.500.000
5.0	Biaya Tak Terduga & Kontingensi (15%)	Contingency buffer for sensor recalibration and protocol adjustments.	Rp 23.500.000
TOTAL	TOTAL (RAB) PILOT PROJECT	Comprehensive 3-month pilot deployment budget across 1 corridor in DKI Jakarta.	Rp 178.500.000

6.1. Three-Stage Disbursement Schedule and Hardware Efficiency Notes

Disbursement is structured into three milestones: Termin 1 (30% at kick-off), Termin 2 (40% upon TraCI-ATCS middleware integration), and Termin 3 (30% upon pilot trial completion and operator training). Furthermore, if Dishub DKI Jakarta provides access to existing roadside CCTV cameras and electronic VMS displays, Item 2.0 can be reduced by Rp 24.000.000, lowering the total budget to Rp 154.500.000.